SENTINELSPORT

AI-Powered Sports Media Protection Platform

Technical Architecture & Build Plan

Hackathon Submission — 10-Day Sprint

Generated: 25 March 2026

Detect	Prove	Act
Triple-layer AI fingerprinting	Blockchain + C2PA provenance	AI-drafted DMCA + alerts

© 2025 SentinelSport — Hackathon Build

1. Executive Summary

Sports organizations globally lose an estimated $28 billion annually to piracy and unauthorized content redistribution. Existing solutions — forensic watermarking vendors like Friend MTS and Irdeto — are enterprise-only contracts priced out of reach for mid-size clubs, federations, and esports organizations. Manual DMCA filing takes hours while live matches are actively being streamed illegally.

SentinelSport is a three-pillar platform that solves the detection, provenance, and response problem in one integrated system:

- Stamp — triple-layer AI fingerprinting + C2PA cryptographic signing + Polygon blockchain anchoring at point of ingestion
- Detect — continuous async crawler across YouTube, Twitter/X, TikTok, and Telegram with FAISS-powered sub-second similarity matching
- Act — AI-generated platform-specific DMCA notices + immutable blockchain evidence package + real-time alert system

Target	Sports leagues, clubs, federations, broadcasters, and esports organizations that cannot afford enterprise-tier anti-piracy vendors but are actively losing revenue to unauthorized redistribution.

Key innovation	The combination of C2PA manifest stripping detection (when a pirate removes the embedded manifest, that removal itself becomes legally admissible evidence of tampering) with on-chain first-publication proof creates a novel, affordable forensic chain that rivals six-figure incumbent tools.

2. Problem Statement

The hackathon brief surfaces three compounding problems:

2.1 The scatter problem

When a league publishes a match highlight, that clip propagates across dozens of platforms within minutes — YouTube, Twitter/X, TikTok, Facebook, Telegram groups, regional forums. The rights holder has no automated way to track this propagation. Manually searching is impossible at scale.

2.2 The detection window problem

For live sports, the piracy damage is front-loaded. If an unauthorized stream runs during the first 30 minutes of a match, it has already served its purpose even if detected and taken down at half-time. Current industry data shows only 2.7% of infringing streams are disrupted within the critical 30-minute window. SentinelSport's continuous polling architecture is designed specifically around this constraint.

2.3 The evidence problem

When a rights holder wants to issue a DMCA notice or pursue legal action, they need: (a) proof they created the content first, (b) proof the infringing copy is their content, and (c) proof the infringer knew and acted deliberately. Existing free tools produce none of these automatically. SentinelSport's blockchain anchor + C2PA manifest + confidence score report addresses all three in one exportable evidence package.

3. System Architecture

SentinelSport is built as a modular, containerized backend with a React frontend. All components communicate via a REST API. Async work (fingerprinting, crawling, matching) runs on Celery workers backed by Redis. The blockchain interaction is a thin web3.py layer — no smart contract complexity bleeds into the application logic.

3.1 Pillar 1 — Stamp (Ingestion & Provenance)

This pillar runs when a sports org uploads their original content. It is the foundation of the entire system — without a registered fingerprint and blockchain anchor, detection has nothing to match against.

Step 1 — Triple-layer fingerprinting

Each uploaded video is processed by three independent fingerprinting algorithms running in parallel:

- Visual fingerprint (videohash library): Wavelet-based perceptual hash computed on frame samples. Survives re-encoding, compression, slight cropping, and color grading. Output: 64-bit hash.
- Audio fingerprint (librosa): Chromagram extracted from the audio track — a 12-dimensional representation of harmonic content. Survives pitch shifting up to ±2 semitones, background noise addition, and lossy compression. Output: averaged chroma vector.

- Scene embedding (OpenCLIP): CLIP ViT-B/32 model encodes keyframes into 512-dimensional semantic vectors. Captures what is happening in the scene, not just pixel patterns. Survives cropping, zooming, text overlays, and logo addition. Output: mean-pooled 512-d vector stored in pgvector.

The three fingerprints are stored independently, enabling the confidence scorer to weight and combine them. The combined SHA-256 hash of all three serves as the master fingerprint ID.

Step 2 — C2PA manifest signing

Using the c2pa-python SDK (the official Python bindings for the Content Authenticity Initiative standard, backed by Adobe, Microsoft, BBC, and Sony), a cryptographically signed manifest is embedded directly into the media file. This manifest records:

- The creator identity (sports org name and API key fingerprint)
- Timestamp of first creation and upload
- SHA-256 hash of the original file
- The SentinelSport platform claim (asserting IP ownership)

The manifest is bound to the file using PKI — the same technology as HTTPS certificates. If any party strips the manifest (which pirates routinely do before re-uploading), the absence of the manifest when cross-referenced against the fingerprint database is itself legally significant evidence of deliberate tampering.

Step 3 — Blockchain anchor

The master fingerprint hash and C2PA manifest hash are written to the MediaRights smart contract on Polygon Amoy testnet (migrating to Polygon mainnet for production). The on-chain record contains:

- fingerprintHash — SHA-256 of the triple fingerprint composite
- c2paManifestHash — SHA-256 of the embedded C2PA manifest
- owner — Ethereum address of the rights holder
- timestamp — block.timestamp, immutable and third-party verified
- mediaTitle — human-readable content identifier

The Polygonscan transaction URL is stored in the violations database and included in every DMCA evidence export. This gives rights holders a court-admissible, blockchain-timestamped proof of first publication that no party — including SentinelSport — can retroactively modify.

3.2 Pillar 2 — Detect (Crawling & Matching)

This pillar runs continuously in the background, dispatched by Celery beat on a configurable schedule (default: every 15 minutes per registered content item).

Platform crawler

yt-dlp is used as the core extraction engine. It is actively maintained, handles platform anti-bot measures, and supports downloading metadata + video without requiring official API access for most platforms. Platform-specific strategies:

- YouTube: Search API (official, free tier) for keyword/description search, then yt-dlp for video extraction on candidates
- Twitter/X: snscrape for tweet search + yt-dlp for video extraction
- TikTok: yt-dlp direct search and extraction
- Telegram: pyrogram library for public channel monitoring (requires phone number registration, documented in setup)

Each crawled candidate video is downloaded at reduced quality (360p) to minimize bandwidth and storage. Only the first 60 seconds are downloaded for matching — sufficient for fingerprint comparison, negligible storage cost.

Matching engine

The fingerprint matching runs in three stages, cheapest-first to minimize compute:

1. Visual hash comparison: XOR distance between stored and candidate pHash. Threshold: Hamming distance ≤ 12 of 64 bits (81%+ similarity). This eliminates >95% of candidates in microseconds.
2. Audio chromagram comparison: Cosine similarity between stored and candidate chroma vectors. Threshold: ≥ 0.85 similarity. Runs only on candidates passing Stage 1.
3. FAISS ANN search: Approximate nearest-neighbour search in 512-dimensional CLIP embedding space using IVF index. Threshold: L2 distance ≤ 0.35. Runs only on candidates passing Stages 1 and 2.

Confidence scoring

For any candidate passing all three stages, a composite confidence score is computed:

Signal	Weight	Rationale
Visual pHash similarity	50%	Hardest to fake without destroying quality
Audio chroma similarity	30%	Survives video re-encoding attacks

Signal	Weight	Rationale
CLIP scene similarity	20%	Catches semantic matches despite heavy processing

Violations with confidence ≥ 85% are automatically stored and trigger alerts. Matches between 70–85% are stored for human review. Below 70% are discarded.

C2PA strip detection

Every crawled video that matches on fingerprint is also checked for C2PA manifest presence using the c2pa-python read API. Three outcomes are possible:

- Manifest present and valid — likely legitimate licensed use, flagged for human review
- Manifest present but tampered — definitive evidence of deliberate modification, automatic high-severity alert
- Manifest absent — strong evidence of deliberate stripping, included in evidence package as proof of tampering intent

3.3 Pillar 3 — Act (Response & Evidence)

AI DMCA notice generator

When a violation is confirmed, the Claude API (claude-sonnet-4-20250514) generates a platform-specific, legally formatted DMCA takedown notice. The prompt includes:

- Infringing URL and platform name
- Confidence score and breakdown per signal
- Blockchain transaction URL (proof of first ownership)
- C2PA manifest status (present/tampered/absent)
- Rights holder identity and contact

The AI generates notices formatted for each platform's specific DMCA intake process (YouTube Copyright Complaint form fields, Twitter/X DMCA web form, TikTok IP Report). This is not a generic template — it is platform-aware legal copy generated fresh for each violation.

Evidence package export

Each violation generates a downloadable evidence package (PDF) containing:

- Screenshot of infringing content at time of detection

- Side-by-side visual comparison: original vs. infringing
- Confidence score report with per-signal breakdown
- Blockchain anchor: Polygonscan link + transaction hash + timestamp
- C2PA manifest diff (original vs. tampered/absent)
- Platform-specific DMCA notice draft

Alert system

High-confidence violations (≥ 85%) trigger immediate alerts via:

- Email: SendGrid API, templated violation summary with direct DMCA link
- Webhook: POST to configurable endpoint (Slack, Teams, PagerDuty)
- Dashboard: Real-time WebSocket push to the React frontend live feed

4. Complete Tech Stack

4.1 Backend

Library / Tool	Version	Purpose
Python	3.11+	Primary backend language
FastAPI	0.111+	Async REST API framework
Uvicorn	latest	ASGI server
Celery	5.3+	Distributed async task queue
Redis	7.x	Celery broker + result backend
PostgreSQL	16.x	Primary relational database
pgvector	0.7+	Vector similarity in PostgreSQL (CLIP embeddings)
SQLAlchemy	2.0+ async	ORM + async session management
asyncpg	latest	Async PostgreSQL driver
Alembic	latest	Database migration management
Pydantic v2	2.x	Data validation + serialization
Docker + Compose	latest	Containerization of all services

4.2 AI / ML / Fingerprinting

Library / Tool	Version	Purpose
videohash	latest	Wavelet-based perceptual video hashing (visual layer)
librosa	0.10+	Audio chromagram extraction (audio layer)
open-clip-torch	latest	CLIP ViT-B/32 scene embeddings (semantic layer)
faiss-cpu	1.8+	ANN similarity search on CLIP vectors
PySceneDetect	0.6+	Shot boundary detection for keyframe extraction
FFmpeg (binary)	6.x	Video decoding and frame extraction
c2pa-python	latest	C2PA manifest signing and verification (CAI SDK)
anthropic	0.28+	Claude API for DMCA notice generation

4.3 Crawler & Platform Integration

Library / Tool	Version	Purpose
yt-dlp	latest	Core video extraction — YouTube, TikTok, Twitter/X
YouTube Data API v3	v3	Keyword/title search for candidate discovery
snscrape	latest	Twitter/X public tweet search
pyrogram	2.x	Telegram public channel monitoring via MTProto API
httpx	latest	Async HTTP client for API calls
aiofiles	latest	Async file I/O for video processing

4.4 Blockchain

Tool	Detail	Purpose
Solidity	0.8.24	Smart contract language — MediaRights.sol
Hardhat	2.x	Contract compilation, testing, and deployment
web3.py	6.x	Python — Polygon RPC interaction from backend
Polygon Amoy testnet	Testnet	Development and demo — free MATIC from faucet
Polygon Mainnet	Mainnet	Production — ~$0.001 per registration tx
ethers.js	6.x	Frontend blockchain tx status and explorer link
Alchemy / Infura	RPC	Polygon RPC endpoint provider

4.5 Frontend

Library / Tool	Version	Purpose
React	18.x	UI framework
Vite	5.x	Build tool and dev server
React Query (TanStack)	5.x	Server state management + auto-refetch
Leaflet + react-leaflet	latest	World map with violation markers
Recharts	2.x	Confidence score charts and violation timeline
Tailwind CSS	3.x	Utility-first styling
shadcn/ui	latest	Accessible component library
jsPDF + html2canvas	latest	Client-side evidence PDF export
ethers.js	6.x	Blockchain tx status display + Polygonscan links
WebSocket (native)	—	Real-time alert feed from FastAPI WS endpoint
axios	1.x	HTTP client for API calls

4.6 DevOps & Infrastructure

Tool	Detail	Purpose
Docker Compose	v2	Local dev: all services in one command
GitHub Actions	CI	Lint, test, Docker build on push
SendGrid	API v3	Transactional violation alert emails
Flower	latest	Celery worker monitoring dashboard
pytest + pytest-asyncio	latest	Backend unit and integration tests
Locust	latest	Load testing for fingerprint API

5. Database Schema

PostgreSQL 16 with the pgvector extension. Four core tables, designed for the read-heavy violation query patterns.

media_assets

Master record for every piece of content registered by a sports org.

Column	Type	Key	Description
id	UUID	PK	Primary identifier
org_id	UUID	FK → orgs	Rights holder organization
title	TEXT	—	Human-readable content title

Column	Type	Key	Description
visual_hash	TEXT	INDEX	64-bit wavelet pHash (hex)
audio_chroma	FLOAT[]	—	12-dim chromagram vector
scene_embedding	VECTOR(512)	IVFFLAT INDEX	CLIP scene embedding (pgvector)
fingerprint_hash	TEXT	UNIQUE INDEX	SHA-256 of composite fingerprint
c2pa_manifest_hash	TEXT	—	SHA-256 of embedded C2PA manifest
blockchain_tx	TEXT	—	Polygon transaction hash
blockchain_ts	BIGINT	—	Block timestamp of on-chain registration
storage_path	TEXT	—	S3 or local path to original file
created_at	TIMESTAMPTZ	—	Upload timestamp

violations

Every detected unauthorized use of registered content.

Column	Type	Key	Description
id	UUID	PK	Primary identifier
asset_id	UUID	FK → media_assets	The original registered content
infringing_url	TEXT	—	URL of the infringing upload
platform	ENUM	INDEX	youtube | twitter | tiktok | telegram | other
visual_score	FLOAT	—	pHash similarity 0.0–1.0
audio_score	FLOAT	—	Chroma cosine similarity 0.0–1.0
scene_score	FLOAT	—	CLIP vector similarity 0.0–1.0
confidence	FLOAT	INDEX	Weighted composite score 0.0–1.0
c2pa_status	ENUM	—	valid | tampered | absent
status	ENUM	INDEX	detected | dmca_sent | taken_down | dismissed
screenshot_path	TEXT	—	Path to detection screenshot
detected_at	TIMESTAMPTZ	INDEX	When the violation was first found
geo_country	CHAR(2)	—	ISO country code from IP geolocation

dmca_notices

AI-generated takedown notices linked to violations.

Column	Type	Key	Description
id	UUID	PK	Primary identifier
violation_id	UUID	FK → violations	Linked violation
platform	ENUM	—	Platform the notice targets
notice_text	TEXT	—	AI-generated notice body
sent_at	TIMESTAMPTZ	—	When email/webhook was triggered
status	ENUM	—	draft | sent | acknowledged | resolved

6. API Endpoints

Method	Endpoint	Description
POST	/api/v1/ingest	Upload original media — runs triple fingerprint + C2PA sign + blockchain anchor
GET	/api/v1/assets	List all registered assets for authenticated org
GET	/api/v1/assets/{id}	Get single asset with fingerprint details + blockchain status
POST	/api/v1/scan/trigger	Manually trigger a full scan cycle for one or all assets
GET	/api/v1/violations	List violations — filter by platform, confidence, status, date range
GET	/api/v1/violations/{id}	Single violation detail with confidence breakdown + screenshot
POST	/api/v1/violations/{id}/dmca	Generate AI DMCA notice for this violation
GET	/api/v1/violations/{id}/evidence	Download PDF evidence package
GET	/api/v1/blockchain/verify/{fingerprint}	Query Polygon contract — returns owner, timestamp, tx hash
GET	/api/v1/stats/dashboard	Aggregate stats for dashboard — violations by platform, trend
WS	/ws/alerts	WebSocket — real-time violation alert push

7. Smart Contract — MediaRights.sol

Minimal Solidity contract deployed on Polygon Amoy testnet. Designed to be cheap (one SSTORE per registration), auditable, and immutable. No upgradeable proxy — simplicity is the security model.

```
// SPDX-License-Identifier: MIT
// Polygon Amoy testnet — MediaRights.sol v1.0
pragma solidity ^0.8.24;

contract MediaRights {
  struct MediaRecord {
    string  fingerprintHash;    // SHA-256 composite fingerprint
    string  c2paManifestHash;   // SHA-256 of C2PA manifest
    address owner;              // Ethereum address of rights holder
    uint256 timestamp;          // block.timestamp — immutable
    string  mediaTitle;         // Human-readable identifier
  }

  mapping(string => MediaRecord) public records;
  event MediaRegistered(string fingerprintHash, address owner, uint256 ts);

  function registerMedia(
    string memory fingerprintHash,
    string memory c2paManifestHash,
    string memory mediaTitle
  ) public {
    require(bytes(records[fingerprintHash].fingerprintHash).length == 0,
            "Already registered");
    records[fingerprintHash] = MediaRecord(
      fingerprintHash, c2paManifestHash, msg.sender,
      block.timestamp, mediaTitle
    );
    emit MediaRegistered(fingerprintHash, msg.sender, block.timestamp);
  }

  function verifyOwnership(string memory fingerprintHash)
    public view
    returns (address, uint256, string memory) {
      MediaRecord memory r = records[fingerprintHash];
      return (r.owner, r.timestamp, r.mediaTitle);
  }
}
```

8. 10-Day Build Plan

Phase 1
Days 1–3
Foundation

Day 1 — Infrastructure setup

Docker Compose with postgres:16, redis:7, fastapi app, celery-worker, celery-beat. Alembic migrations. Health check endpoints. GitHub repo + .env template.

Day 2 — Triple fingerprint engine

videohash + librosa + OpenCLIP pipeline. /api/v1/ingest endpoint. Store fingerprints in PostgreSQL + FAISS index. Unit tests for each fingerprint layer. Target: 5 min per video max.

Day 3 — C2PA + FAISS

c2pa-python signing integrated into ingestion flow. FAISS IVF index built from pgvector data. /api/v1/blockchain/verify stub. Milestone: upload a video, get a fingerprint JSON + C2PA manifest hash.

Phase 2
Days 4–6
Detection

Day 4 — Platform crawler

yt-dlp async Celery tasks for YouTube + TikTok. YouTube Data API v3 search integration. Download candidates at 360p, first 60 seconds only. Rate limiting + retry logic.

Day 5 — Matching engine + confidence scorer

3-stage matching pipeline (pHash → chroma → FAISS). Weighted confidence formula. Violations table populated on match. /api/v1/violations endpoint live.

Day 6 — C2PA strip detection + violations polish

c2pa-python read on crawled candidates. c2pa_status field populated. Celery beat scheduler for automated scan cycles. Integration test: end-to-end upload → detect → violation.

Phase 3
Days 7–8
Chain + AI

Day 7 — Blockchain layer

MediaRights.sol compiled and deployed to Polygon Amoy via Hardhat. web3.py integration in ingestion endpoint. blockchain_tx + blockchain_ts stored on media_assets. /api/v1/blockchain/verify endpoint live. Polygonscan link generation.

Day 8 — Claude DMCA generator + alert system

Anthropic API integration. Platform-aware prompt templates for YouTube / Twitter / TikTok DMCA formats. /api/v1/violations/{id}/dmca endpoint. SendGrid email alerts on high-confidence match. WebSocket push for real-time feed.

Phase 4
Days 9–10
UI + Demo

Day 9 — React dashboard

Leaflet world map with violation markers colored by severity. Violation card list with confidence breakdown bar chart. DMCA generator panel. Blockchain verify panel with Polygonscan link. Evidence PDF export button. Live alert feed via WebSocket.

Day 10 — Demo prep + submission

Seed demo dataset (10 registered assets, 25 pre-detected violations across 4 platforms). Rehearse 4-minute demo script. Write README with setup instructions. Record 2-minute backup video demo. Submit.

9. Team Responsibilities

Role	Days	Owns
ML / CV Engineer	1–6 primary, 7–10 support	Fingerprint engine (videohash, librosa, CLIP), FAISS index, confidence scorer, C2PA strip detector, false positive tuning
Backend Engineer	1–8 primary	FastAPI structure, Celery workers, database schema + migrations, all API endpoints, crawler integration, SendGrid + WebSocket
Frontend Engineer	1–2 scaffold, 7–10 primary	React app, Leaflet map, Recharts, violation cards, DMCA panel, evidence PDF export, WebSocket live feed, Tailwind styling
Data / Crawler Engineer	3–6 primary, 7–10 support	yt-dlp async workers, YouTube API, snscrape, pyrogram Telegram, demo dataset creation, rate limiting logic, Celery beat scheduling
Blockchain (any member)	7 focused	Hardhat setup, MediaRights.sol deploy, web3.py integration, Polygonscan link generation — estimated 8–10 hours total

10. Novelty & Competitive Differentiation

What makes SentinelSport genuinely novel

1. Triple-layer fingerprint fusion

Every competitor uses one signal — either visual hash or audio fingerprint, never both, never combined with semantic embeddings. SentinelSport fuses all three with explainable per-signal weights. A pirate must defeat all three simultaneously — re-encode the video, shift the audio pitch, AND change what is happening in the scene — to avoid detection. This is computationally impossible without destroying the content's value.

2. C2PA manifest stripping as evidence

The C2PA standard was designed for AI-generated content provenance (Adobe, BBC, Microsoft) but has never been applied to sports rights enforcement. SentinelSport is the first system to treat the absence of a C2PA manifest — when cross-referenced against a fingerprint match — as affirmative evidence of deliberate tampering. This is a novel forensic argument: the pirate's removal of the credential becomes the crime.

3. Blockchain as legal first-publication proof

Existing watermarking tools (Friend MTS, Nagravision) keep ownership records in proprietary databases that a defense attorney can challenge. A Polygon blockchain record is a decentralized, third-party-verified timestamp that no single party controls. The Polygonscan link in a DMCA notice is harder to dismiss than a vendor's internal log.

4. Affordable integrated platform

The three most capable existing tools — Sportian Piracy Guard, Friend MTS watermarking, and rights-holder legal teams — are sold as separate, expensive services. SentinelSport integrates all three capabilities into one platform, targeted at the underserved market of mid-size sports organizations, esports leagues, and content federations that cannot afford six-figure enterprise contracts.

Market gap — the numbers

Metric	Incumbents	SentinelSport (target)	Improvement
Streams disrupted in 30-min window	2.7%	25%+	9× better
DMCA notice generation time	Hours (manual)	< 60 seconds	100× faster
Platforms monitored	Partner-only	Any public platform	Unbounded
Entry price point	$50,000+ / year	SaaS tier pricing	Accessible to SMB
Ownership proof mechanism	Proprietary DB	Immutable blockchain	Court-defensible

11. Demo Script — 4-Minute Flow

Rehearse this exactly. Every step is visual, clickable, and tells a story. Do not improvise.

Min	Moment	What you show
0:00	Upload	Drag a 30-second sports clip into the dashboard. Show the three fingerprint scores computing in real time. Show the Polygon transaction confirming on Amoy explorer (~2 sec).
1:00	Violation appears	Switch to the pre-seeded violation card. Show infringing URL, platform badge, upload timestamp, and the confidence breakdown: Visual 94%, Audio 88%, Scene 91%, Overall 92%.
2:00	C2PA smoking gun	Click 'Verify provenance'. Show the original file manifest vs. the crawled file's absent manifest. One line: 'Copyright metadata was deliberately removed — this is evidence of tampering.'

Min	Moment	What you show
2:45	Blockchain proof	Click the Polygonscan link. Show the on-chain record: owner address, timestamp (before the violation), transaction hash. 'This is a court-admissible timestamp that no one can backdate.'
3:20	One-click DMCA	Click 'Generate Notice'. Claude API generates a YouTube-formatted takedown in 3 seconds. Show the email preview. Click 'Download Evidence Package' — PDF exports with all of the above.
3:50	Close	Return to world map showing 25 violations across 4 platforms. 'SentinelSport found these in the last 24 hours — automatically, without a single manual search.'

12. Risk Register

Risk	Severity	Mitigation
yt-dlp blocked by platform during demo	High	Pre-download all demo video candidates on Day 9. Demo uses pre-seeded data, not live crawls.
CLIP model too slow on CPU (>5 min/video)	High	Use CLIP ViT-B/32 (smallest variant). Pre-compute all embeddings for demo dataset. GPU fallback: Google Colab for fingerprint generation.
Polygon testnet congestion / faucet issues	Medium	Request testnet MATIC on Day 6 — not Day 7. Keep a backup wallet with pre-funded MATIC. Fallback: mock blockchain response for demo.
c2pa-python build fails on target OS	Medium	Test on Day 1. It requires Rust toolchain. Docker image pre-installs Rust — document this clearly.
False positives on confidence scorer	Medium	Set 85% threshold conservatively. Add human-review queue for 70–85% range. Tune thresholds on Day 6 using real crawled candidates.
Claude API rate limits during demo	Low	Pre-generate all DMCA notices for demo violations on Day 9. Cache them. Show live generation only once.

13. Environment Setup Checklist

Required API keys (obtain on Day 1)

- YouTube Data API v3 key — Google Cloud Console, free tier: 10,000 units/day
- Anthropic API key — claude.ai/settings, needed for DMCA generation
- SendGrid API key — free tier: 100 emails/day

- Alchemy or Infura API key — Polygon RPC endpoint, free tier sufficient
- Telegram API ID + API hash — my.telegram.org, for pyrogram (requires phone number)

Docker Compose services

- postgres:16-alpine — port 5432, pgvector extension enabled
- redis:7-alpine — port 6379, Celery broker
- fastapi-app — port 8000, mounts model cache volume
- celery-worker — same image as fastapi-app, runs workers
- celery-beat — same image, runs scheduler
- flower — port 5555, Celery monitoring UI
- react-frontend — port 3000, Vite dev server

Day 1 commands — get the stack running

```
git clone https://github.com/your-org/sentinelsport
cp .env.example .env   # fill in all API keys
docker compose up -d   # start all services
docker compose exec fastapi-app alembic upgrade head   # run migrations
curl -X POST http://localhost:8000/api/v1/ingest \
     -F "file=@test_clip.mp4" -F "title=Test Highlight"
# Expected: { fingerprint_hash, blockchain_tx, c2pa_manifest_hash }
```

14. Success Metrics

These are the numbers to hit before Day 10 submission. They are demonstrable, not aspirational.

Metric	Target	How to verify
Fingerprint generation time (60-sec clip)	< 90 seconds	Postman timer on /ingest
Matching accuracy on test set (10 known pairs)	≥ 90% true positive rate	Manual test on Day 6
False positive rate (10 unrelated videos)	< 5%	Manual test on Day 6
Blockchain registration latency	< 10 seconds	Timed in Postman
DMCA notice generation time	< 5 seconds	Timed in Postman
Evidence PDF export	Working, < 3 MB	Manual download test
Demo dataset seeded	10 assets, 25 violations	Dashboard shows counts

SentinelSport — Built in 10 days. Designed for the long game.

Detect. Prove. Act. Repeat.